

پروژه پاتوق (پاتوق)
سامانه مدیریت و رزرو کافه و رستوران

گزارش اسپرینت اول
زیرساخت و پایه گذاری

درس: طراحی نرم افزار
مدت اسپرینت: ۲ هفته
تاریخ تهیه: تیر ۱۴۰۵

این گزارش پوشش دهی اسپرینت اول پروژه پاتوق را شرح می دهد که در آن زیرساخت اصلی بک اند و فرانت اند، سیستم احراز هویت و موجودیت های اصلی دامنه پیاده سازی شدند.

فهرست مطالب

۲	۱ هدف اسپرینت
۲	۱.۱ تعریف «انجام شده» (Definition of Done)
۳	۲ Sprint Backlog
۴	۳ پیاده سازی بک اند
۴	۱.۳ معماری تمیز (Clean Architecture)
۴	۲.۳ موجودیت های دامنه
۴	۳.۳ پیکربندی پایگاه داده
۴	۴.۳ سیستم احراز هویت
۴	۱.۴.۳ جریان OTP
۵	۲.۴.۳ ویژگی های امنیتی OTP
۵	۳.۴.۳ ساختار JWT
۵	۵.۳ Endpoint های پیاده سازی شده
۶	۴ پیاده سازی فرانت اند
۶	۱.۴ راه اندازی پروژه
۶	۲.۴ سیستم مسیریابی
۶	۳.۴ صفحات پیاده سازی شده
۶	۱.۳.۴ صفحه انتخاب نقش (RoleSelection)
۶	۲.۳.۴ صفحه ورود (LoginPage)
۷	۳.۳.۴ صفحه تأیید OTP (OTPVerification)
۷	۴.۳.۴ صفحه ثبت نام (RegisterPage)
۷	۴.۴ مدیریت وضعیت احراز هویت
۸	۵ چالش ها و راه حل ها
۸	۱.۵ چالش ۱: خطای BCrypt با هش خالی
۸	۲.۵ چالش ۲: پیکربندی CORS
۸	۳.۵ چالش ۳: مدیریت وضعیت IsNewUser
۹	۶ بازبینی و گذر اسپرینت
۹	۱.۶ Review Sprint
۹	۲.۶ Retrospective Sprint

۱ هدف اسپرینت

هدف اصلی اسپرینت اول، ایجاد پایه‌های محکم معماری سیستم و پیاده‌سازی جریان احراز هویت کامل است. در پایان این اسپرینت باید کاربر بتواند از طریق OTP یا رمز عبور وارد سیستم شود و توکن‌های دسترسی دریافت کند.

۱.۱ تعریف «انجام‌شده» (Definition of Done)

- کد بدون خطای کامپایل
- API مستندسازی‌شده در Swagger
- Migration های پایگاه داده اعمال‌شده
- صفحات فرانت‌اند با UI کارکرد واقعی (نه mock)
- CORS صحیح پیکربندی‌شده

Sprint Backlog ۲

آیتم	توضیح	مسئول	وضعیت
راه اندازی پروژه بک اند	معماری تمیز، لایه بندی	بک اند	انجام شد
تعریف موجودیت های دامنه	کلاس های Domain	بک اند	انجام شد
پیگر بندی EF Core	DbContext، نگاشت ها	بک اند	انجام شد
پیگر بندی Redis	OTP و کش	بک اند	انجام شد
Docker Compose	Redis + PostgreSQL	بک اند	انجام شد
سرویس OTP	ارسال و تأیید کد	بک اند	انجام شد
سرویس JWT	صدور توکن دسترسی + تازه سازی	بک اند	انجام شد
AuthController	Endpoint های احراز هویت	بک اند	انجام شد
Middleware خطا	فرمت RFC 7807	بک اند	انجام شد
راه اندازی پروژه فرانت اند	Vite + React + TypeScript	فرانت اند	انجام شد
سیستم مسیریابی	React Router 7	فرانت اند	انجام شد
صفحه انتخاب نقش	RoleSelection	فرانت اند	انجام شد
صفحه ورود	LoginPage	فرانت اند	انجام شد
صفحه تأیید OTP	OTPVerification	فرانت اند	انجام شد
صفحه ثبت نام	RegisterPage	فرانت اند	انجام شد
AuthContext	مدیریت وضعیت احراز هویت	فرانت اند	انجام شد

۳ پیاده‌سازی بک‌اند

۱.۳ معماری تمیز (Clean Architecture)

پروژه بک‌اند بر اساس معماری تمیز با چهار لایه اصلی ساختاردهی شد:

Patogh.Domain: موجودیت‌های دامنه، Enum ها و اینترفیس‌های پایه. هیچ وابستگی خارجی ندارد.

Patogh.Application: منطق کسب‌وکار با الگوی CQRS + MediatR. فقط به Domain وابسته است.

Patogh.Infrastructure: سرویس‌های خارجی: JWT، OTP، ارسال پیامک، آپلود فایل.

Patogh.Persistence: DbContext، Migration ها و پیکربندی EF Core.

Patogh.API: کنترلرها، Middleware ها و تنظیمات ASP.NET Core.

۲.۳ موجودیت‌های دامنه

در این اسپرینت موجودیت‌های اصلی دامنه تعریف شدند:

موجودیت	فیلدهای اصلی	توضیح
User	PhoneNumber, PasswordHash, Role, DisplayName, AvatarUrl	کاربر سیستم
Restaurant	Name, Description, Location, FoodType, Status, OwnerId	رستوران
RestaurantTable	TableNumber, Capacity, RestaurantId	میز رستوران
MenuItem	Name, Description, Price, ImageUrl, RestaurantId	آیتم منو
Reservation	CustomerId, TableId, Date, StartTime, EndTime, Status	رزرو
RefreshToken	Token, UserId, ExpiresAt, IsRevoked	توکن تازه‌سازی
UserFavorite	UserId, RestaurantId	علاقه‌مندی
MediaAsset	Url, FileName, ContentType	فایل رسانه‌ای

۳.۳ پیکربندی پایگاه داده

- PostgreSQL: پایگاه داده اصلی از طریق Docker Compose راه‌اندازی شد.
- EF Core 8: برای ORM و مدیریت Migration ها.
- Redis: برای ذخیره‌سازی کدهای OTP (با ۲ دقیقه TTL) و کش.
- Docker Compose: هر دو سرویس PostgreSQL و Redis در یک شبکه Docker اجرا می‌شوند.

۴.۳ سیستم احراز هویت

۱.۴.۳ جریان OTP

۱. کاربر شماره موبایل را ارسال می‌کند (POST /api/auth/send-otp)

۲. سرور یک کد ۶ رقمی تصادفی تولید و در Redis ذخیره می‌کند (کلید: otp:{phone})
۳. کد از طریق سرویس پیامک ارسال می‌شود
۴. کاربر کد را برای تأیید ارسال می‌کند (POST /api/auth/verify-otp)
۵. پس از تأیید، کد از Redis حذف می‌شود (یکبار مصرف)
۶. توکن دسترسی (JWT) و توکن تازه‌سازی صادر می‌شود

۲.۴.۳ ویژگی‌های امنیتی OTP

- اعتبار ۲ دقیقه‌ای (Redis در TTL)
- یکبار مصرف: پس از تأیید موفق بلافاصله حذف می‌شود
- OTP در لاگ‌های سیستم ثبت نمی‌شود (امنیت)
- محدودیت ۵ درخواست در دقیقه برای جلوگیری از Brute Force

۳.۴.۳ ساختار JWT

- توکن دسترسی: عمر کوتاه (۱۵ دقیقه)، حاوی UserId، Role، PhoneNumber
- توکن تازه‌سازی: عمر بلند (۷ روز)، ذخیره در پایگاه داده
- Token Rotation: هر بار استفاده از توکن تازه‌سازی، توکن جدید صادر می‌شود

۵.۳ Endpoint های پیاده‌سازی شده

متد	مسیر	توضیح	احراز هویت
POST	/api/auth/send-otp	ارسال کد OTP	بدون نیاز
POST	/api/auth/verify-otp	تأیید OTP و دریافت توکن	بدون نیاز
POST	/api/auth/login	ورود با رمز عبور	بدون نیاز
POST	/api/auth/register	ثبت‌نام با رمز عبور	بدون نیاز
POST	/api/auth/refresh	تجدید توکن دسترسی	بدون نیاز
POST	/api/auth/revoke	باطل کردن توکن (خروج)	نیاز دارد

۴ پیاده‌سازی فرانت‌اند

۱.۴ راه‌اندازی پروژه

پروژه فرانت‌اند با Vite 6.3.5 + React 18 + TypeScript راه‌اندازی شد. پکیج‌های اصلی نصب‌شده:

- React Router 7: مسیریابی سمت کلاینت
- Tailwind CSS v4: استایل‌دهی با @tailwindcss/vite
- shadcn/ui + Radix UI: کامپوننت‌های UI
- React Hook Form: مدیریت فرم‌ها
- Sonner: نمایش اعلان‌های toast
- Lucide React: آیکون‌ها
- date-fns: کار با تاریخ

۲.۴ سیستم مسیریابی

مسیریابی پروژه در فایل App.tsx تعریف شده است و شامل سه گروه اصلی است: مسیرهای عمومی، مسیرهای احراز هویت و مسیرهای محافظت‌شده بر اساس نقش. کامپوننت ProtectedRoute از دسترسی کاربران بدون نقش مناسب به صفحات محافظت‌شده جلوگیری می‌کند.

۳.۴ صفحات پیاده‌سازی‌شده

۱.۳.۴ صفحه انتخاب نقش (RoleSelection)

اولین صفحه در جریان ورود، دو دکمه برای انتخاب نقش مشتری یا مدیر رستوران نمایش می‌دهد و کاربر را به صفحه ورود مناسب هدایت می‌کند.

۲.۳.۴ صفحه ورود (LoginPage)

- دو حالت: ورود با OTP (پیش‌فرض) و ورود با رمز عبور
- اعتبارسنجی فرمت شماره موبایل ایرانی
- ارسال درخواست OTP و هدایت به صفحه تأیید
- نمایش پیام‌های خطای فارسی

۳.۳.۴ صفحه تأیید OTP (OTPVerification)

- اینپوت ۶ خانه‌ای برای کد OTP
- تایمر شمارش معکوس ۲ دقیقه‌ای
- امکان درخواست ارسال مجدد کد
- هدایت خودکار پس از تأیید موفق بر اساس نقش

۴.۳.۴ صفحه ثبت‌نام (RegisterPage)

فرم ثبت‌نام با شماره موبایل، رمز عبور و تأیید رمز عبور. اعتبارسنجی تمام فیلدها با React Hook Form.

۴.۴ مدیریت وضعیت احراز هویت

AuthContext برای مدیریت وضعیت ورود کاربر در تمام صفحات ایجاد شد:

- ذخیره توکن دسترسی در localStorage
- ذخیره اطلاعات کاربر (نقش، شماره، نام)
- تابع خروج از سیستم
- بارگذاری خودکار وضعیت از localStorage پس از رفرش صفحه

۵ چالش‌ها و راه‌حل‌ها

۱.۵ چالش ۱: خطای BCrypt با هش خالی

مشکل: هنگام ثبت‌نام کاربر بدون رمز عبور (ورود با OTP)، فراخوانی BCrypt.Verify با رشته خالی خطا می‌داد.

راه‌حل: افزودن بررسی string.IsNullOrEmpty(PasswordHash) قبل از فراخوانی BCrypt. کاربران onlyOTP- مقدار PasswordHash خالی دارند و ورود با رمز عبور برای آن‌ها امکان‌پذیر نیست.

۲.۵ چالش ۲: پیکربندی CORS

مشکل: فرانت‌اند روی پورت 5173 اجرا می‌شد اما CORS سرور این پورت را مجاز نمی‌دانست.

راه‌حل: پیکربندی CORS با افزودن http://localhost:5173 به لیست AllowedOrigins در appsettings.json. همچنین AllowCredentials() برای ارسال کوکی‌های توکن تازه‌سازی فعال شد.

۳.۵ چالش ۳: مدیریت وضعیت IsNewUser

مشکل: تشخیص اینکه کاربر پس از تأیید OTP باید به صفحه تکمیل پروفایل هدایت شود یا به داشبورد.

راه‌حل: فیلد HasIncompleteProfile در پاسخ verify-otp اضافه شد. فرانت‌اند بر اساس این فیلد تصمیم‌گیری می‌کند.

۶ بازبینی و گذر اسپرینت

۱.۶ Review Sprint

- تمامی Endpoint های احراز هویت پیاده سازی و تست شدند
- جریان کامل OTP از ارسال تا دریافت توکن کار می کند
- صفحات فرانت اند به API واقعی وصل شدند
- Docker Compose برای محیط توسعه پیکربندی شد

۲.۶ Retrospective Sprint

چه چیزی خوب بود:

- معماری تمیز از ابتدا پیاده سازی شد که نگهداری کد را ساده کرد
- استفاده از MediatR کدهای کنترلر را کوچک و تمیز نگه داشت
- Docker Compose راه اندازی محیط توسعه را سریع کرد

چه چیزی می توانست بهتر باشد:

- پیکربندی CORS باید زودتر انجام می شد تا تأخیر در اتصال فرانت به بک اند کاهش می یافت
- باید از ابتدا برای HasIncompleteProfile برنامه ریزی می شد

اقدامات بهبود برای اسپرینت بعدی:

- تعریف مشخص تر API contract قبل از شروع پیاده سازی
- نوشتن Integration Test برای Endpoint های اصلی